

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

**Дисциплина:
«Программирование»**

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

«Строки и ввод-вывод»

вариант - 8-1-1-4

Выполнил:

студент гр. N3151

Агакарян Арам Арманович



(подпись)

Проверил:

инженер ФБИТ

Волков Александр Григорьевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

Дата и время

В формате HH:MM:SS dd.mm.yyyy

Примеры:

12:24:01 01.02.2001, 00:00:00 31.12.2023

Задание: -b=M Опция -b задает номер начальной строки (строки нумеруются с единицы). Преобразование выполняется, начиная со строки M. Если опция -b не указана, то преобразование начинается с первой строки.

-e=N Опция -e задает номер конечной строки. Преобразование выполняется до строки N включительно. Если опция -e не указана, преобразование продолжается до последней строки.

Makefile

```
← → prg3aaaN3151.c inf2aaaN3151.S Makefile
prg3aaaN3151/Makefile> .PHONY: all clean rebuild

1  PROGRAM = prg3aaaN3151
2
3  CC = gcc
4  CFLAGS = -Wall -Wextra -Werror -std=c99 -pedantic
5
6  SOURCES = prg3aaaN3151.c
7  OBJECTS = $(SOURCES:.c=.o)
8
▷ 9  all: $(PROGRAM)
10
▷ 11  $(PROGRAM): $(OBJECTS)
12      $(CC) $(CFLAGS) -o $(PROGRAM) $(OBJECTS)
13
▷ 14  %.o: %.c
15      $(CC) $(CFLAGS) -c $< -o $@
16
▷ 17  clean:
18      rm -f $(OBJECTS) $(PROGRAM)
19
▷ 20  rebuild: clean all
21
▷ 22  .PHONY: all clean rebuild
```

примеры работы программ

```
~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
```

```
14:02:17 $ ./prg3aaaN3151 -v
```

```
Студент: [Агакарян Арам Арманович], гр. [N3151]
```

```
Вариант: 8-1-1-4
```

```
~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
```

```
14:03:52 $ ./prg3aaaN3151 test.txt
```

```
Системный журнал:
```

```
*00:00:00 01.01.2024* - Сервер запущен
```

```
*12:30:00 15.03.2024* - Пользователь вошел
```

```
*23:59:59 31.12.2024* - Конец года
```

```
~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
```

```
14:03:55 $ ./prg3aaaN3151 -c test.txt
```

```
Системный журнал:
```

```
00:00:00 01.01.2024 - Сервер запущен
```

```
12:30:00 15.03.2024 - Пользователь вошел
```

```
23:59:59 31.12.2024 - Конец года
```

```
~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
```

```
14:28:05 $ ./prg3aaaN3151 -c /home/harry/Downloads/"Telegram Desktop"/aram-prog3/test2.txt
```

```
Проверка валидации:
```

```
00:00:00 01.01.2000 - Корректная дата
```

```
23:59:59 31.12.2999 - Корректная дата
```

```
25:00:00 01.01.2024 - Неверные часы (>23)
```

```
12:60:00 01.01.2024 - Неверные минуты (>59)
```

```
12:30:60 01.01.2024 - Неверные секунды (>59)
```

```
12:30:00 32.01.2024 - Неверный день (>31)
```

```
12:30:00 01.13.2024 - Неверный месяц (>12)
```

```
12:30:00 00.01.2024 - Неверный день (=0)
```

```
~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
```

```
14:28:22 $ ./prg3aaaN3151 -c /home/harry/Downloads/"Telegram Desktop"/aram-prog3/test3.txt
```

```
Встреча в 09:00:00 15.01.2024, а затем еще одна в 14:30:00 15.01.2024.
```

```
Дата начала проекта: 10:00:00 01.02.2024
```

```
~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
```

```
14:28:40 $ ./prg3aaaN3151 -c /home/harry/Downloads/"Telegram Desktop"/aram-prog3/test4.txt
```

```
Событие произошло в 12:30:00
```

```
01.01.2024 по московскому времени.
```

```
А это корректная дата: 15:45:30 20.12.2024.
```

```
~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
14:28:45 $ ./prg3aaaN3151 -c /home/harry/Downloads/"Telegram Desktop"/aram-prog3/test5.txt
Строка 1: 10:00:00 01.01.2024
Строка 2: 11:00:00 02.01.2024
Строка 3: 12:00:00 03.01.2024
Строка 4: 13:00:00 04.01.2024
Строка 5: 14:00:00 05.01.2024
Строка 6: 15:00:00 06.01.2024

~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
14:28:49 $ ./prg3aaaN3151 -c /home/harry/Downloads/"Telegram Desktop"/aram-prog3/test6.txt
=== Журнал событий ===

ИНФОРМАЦИЯ: Система инициализирована в 08:15:30 10.12.2024
ПРЕДУПРЕЖДЕНИЕ: Низкий уровень памяти
ОШИБКА: Сбой соединения в 08:16:45 10.12.2024

Следующая проверка: 09:00:00 11.12.2024
Формат даты должен быть строгим!

Неправильный формат: 8:15:30 10.12.2024 (нет ведущего нуля)
Неправильный формат: 08:15:3 10.12.2024 (не хватает цифры)

ИНФОРМАЦИЯ: Проблема решена в 10:30:00 10.12.2024

~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
14:37:52 $ ./prg3aaaN3151 -c abc
Ошибка: не удалось открыть файл abc

~/Desktop/prg3aaaN3151 via C v13.3.0-gcc
14:43:02 $ ./prg3aaaN3151 -x test.txt
Ошибка: неподдерживаемая опция -x
```

исходный текст программы

```
#include <stdio.h> #include <stdlib.h> #include <string.h>
```

```
/* Структура для хранения параметров программы */
```

```
typedef struct { int show_version; / Флаг опции -v / int use_color; / Флаг опции -c / int single_line_only; /
Флаг опции -n / int begin_line; / Начальная строка (-b), -1 если не задана / int end_line; / Конечная
строка (-e), -1 если не задана */ char input_file; / Имя входного файла или NULL */ char output_file; /
Имя выходного файла или NULL */ } Options;
```

```
/* Прототипы функций */ void print_version(void); int parse_options(int argc, char *argv[], Options
*opts); int is_digit_char(char c); int is_valid_date_time(const char *str, int pos); void process_file(FILE
*input, FILE *output, Options *opts); void process_line(const char *line, FILE *output, Options *opts, int
line_num);
```

```
/* Глобальные константы */ #define ANSI_BLUE "\033[34m" #define ANSI_RESET "\033[0m"
```

```
int main(int argc, char *argv[]) { Options opts = {0, 0, 0, -1, -1, NULL, NULL}; FILE *input = stdin; FILE
*output = stdout; int exit_code = 0;
```

```
/* Парсим аргументы командной строки */
```

```
if (parse_options(argc, argv, &opts) != 0) {
    return 1;
}
```

```

/* Если нужно показать версию - показываем и выходим */
if (opts.show_version) {
    print_version();
    return 0;
}

/* Открываем входной файл, если указан */
if (opts.input_file != NULL) {
    input = fopen(opts.input_file, "r");
    if (input == NULL) {
        fprintf(stderr, "Ошибка: не удалось открыть файл %s\n", opts.input_file);
        return 2;
    }
}

/* Открываем выходной файл, если указан */
if (opts.output_file != NULL) {
    output = fopen(opts.output_file, "w");
    if (output == NULL) {
        fprintf(stderr, "Ошибка: не удалось создать файл %s\n", opts.output_file);
        if (input != stdin) fclose(input);
        return 2;
    }
}

/* Обработываем файл */
process_file(input, output, &opts);

/* Закрываем файлы */
if (input != stdin) fclose(input);
if (output != stdout) fclose(output);

return exit_code;

}

/* Функция вывода информации о версии */ void print_version(void) { printf("Студент: Агакарян\n"); printf("Арам Арманович, гр. N3151\n"); printf("Вариант: 8-1-1-4\n"); }

/* Функция парсинга опций командной строки */ int parse_options(int argc, char *argv[], Options *opts) { int i; int file_arg_count = 0;

/* Проходим по всем аргументам */
for (i = 1; i < argc; i++) {
    /* Если аргумент начинается с '-', это опция */
    if (argv[i][0] == '-') {
        if (strcmp(argv[i], "-v") == 0) {
            opts->show_version = 1;
        } else if (strcmp(argv[i], "-c") == 0) {
            opts->use_color = 1;
        } else if (strcmp(argv[i], "-n") == 0) {
            opts->single_line_only = 1;
        } else if (strncmp(argv[i], "-b=", 3) == 0) {
            /* Парсим число после -b= */

```

```

    char *num_str = argv[i] + 3;
    char *endptr;
    long val = strtol(num_str, &endptr, 10);
    if (*endptr != '\0' || val < 1) {
        fprintf(stderr, "Ошибка: %s не является допустимым числом для -b\n", num_str);
        return 1;
    }
    opts->begin_line = (int)val;
} else if (strncmp(argv[i], "-e=", 3) == 0) {
    /* Парсим число после -e= */
    char *num_str = argv[i] + 3;
    char *endptr;
    long val = strtol(num_str, &endptr, 10);
    if (*endptr != '\0' || val < 1) {
        fprintf(stderr, "Ошибка: %s не является допустимым числом для -e\n", num_str);
        return 1;
    }
    opts->end_line = (int)val;
} else {
    fprintf(stderr, "Ошибка: неподдерживаемая опция %s\n", argv[i]);
    return 1;
}
}
} else {
    /* Это не опция, значит имя файла */
    if (file_arg_count == 0) {
        opts->input_file = argv[i];
        file_arg_count++;
    } else if (file_arg_count == 1) {
        opts->output_file = argv[i];
        file_arg_count++;
    } else {
        fprintf(stderr, "Ошибка: слишком много аргументов\n");
        return 1;
    }
}
}
}

/* Проверка корректности диапазона строк */
if (opts->begin_line != -1 && opts->end_line != -1) {
    if (opts->begin_line > opts->end_line) {
        fprintf(stderr, "Ошибка: начальная строка (%d) больше конечной (%d)\n",
            opts->begin_line, opts->end_line);
        return 1;
    }
}
}

return 0;

}

/* Проверка, является ли символ цифрой */ int is_digit_char(char c) { return c >= '0' && c <= '9'; }

/* Проверка корректности даты и времени в позиции pos */ int is_valid_date_time(const char *str, int
pos) { int i = pos; int len = strlen(str);

```

```

/* Проверяем, что есть достаточно места для полной даты */
if (pos + 19 > len) return 0;

/* Проверяем формат HH:MM:SS dd.mm.yyyy */
/* HH - часы (00-23) */
if (!is_digit_char(str[i]) || !is_digit_char(str[i+1])) return 0;
int hours = (str[i] - '0') * 10 + (str[i+1] - '0');
if (hours > 23) return 0;
i += 2;

/* : */
if (str[i] != ':') return 0;
i++;

/* MM - минуты (00-59) */
if (!is_digit_char(str[i]) || !is_digit_char(str[i+1])) return 0;
int minutes = (str[i] - '0') * 10 + (str[i+1] - '0');
if (minutes > 59) return 0;
i += 2;

/* : */
if (str[i] != ':') return 0;
i++;

/* SS - секунды (00-59) */
if (!is_digit_char(str[i]) || !is_digit_char(str[i+1])) return 0;
int seconds = (str[i] - '0') * 10 + (str[i+1] - '0');
if (seconds > 59) return 0;
i += 2;

/* Пробел или перенос строки */
if (str[i] != ' ' && str[i] != '\n') return 0;
i++;

/* dd - день (01-31) */
if (!is_digit_char(str[i]) || !is_digit_char(str[i+1])) return 0;
int day = (str[i] - '0') * 10 + (str[i+1] - '0');
if (day < 1 || day > 31) return 0;
i += 2;

/* . */
if (str[i] != '.') return 0;
i++;

/* mm - месяц (01-12) */
if (!is_digit_char(str[i]) || !is_digit_char(str[i+1])) return 0;
int month = (str[i] - '0') * 10 + (str[i+1] - '0');
if (month < 1 || month > 12) return 0;
i += 2;

/* . */
if (str[i] != '.') return 0;
i++;

/* yyyy - год (4 цифры) */

```

```

if (!is_digit_char(str[i]) || !is_digit_char(str[i+1]) ||
    !is_digit_char(str[i+2]) || !is_digit_char(str[i+3])) return 0;

return 1; /* Формат корректен */

}

/* Обработка всего файла */ void process_file(FILE *input, FILE *output, Options *opts) { char *buffer
= NULL; size_t total_read = 0; char chunk[4096]; size_t bytes_read;

/* Читаем весь файл в буфер */
while ((bytes_read = fread(chunk, 1, sizeof(chunk), input)) > 0) {
    char *new_buffer = realloc(buffer, total_read + bytes_read + 1);
    if (new_buffer == NULL) {
        free(buffer);
        return;
    }
    buffer = new_buffer;
    memcpy(buffer + total_read, chunk, bytes_read);
    total_read += bytes_read;
}

if (buffer != NULL) {
    buffer[total_read] = '\0';

    /* Обработываем содержимое */
    process_line(buffer, output, opts, 0);

    free(buffer);
}

}

/* Обработка текста */ void process_line(const char *text, FILE *output, Options *opts, int unused) { int i
= 0; int len = strlen(text); int line_num = 1;

(void)unused; /* Параметр не используется */

while (i < len) {
    /* Отслеживаем номер строки */
    if (text[i] == '\n') {
        line_num++;
    }

    /* Проверяем, начинается ли здесь дата-время */
    /* Длина шаблона HH:MM:SS dd.mm.yyyy = 19 символов */
    if (i + 19 <= len && is_valid_date_time(text, i)) {
        /* Определяем, в каком диапазоне строк находится дата */
        int date_line_start = line_num;
        int j;

        /* Проверяем опции -b и -e */
        int should_process = 1;
        if (opts->begin_line != -1 && date_line_start < opts->begin_line) {

```

```

    should_process = 0;
}
if (opts->end_line != -1 && date_line_start > opts->end_line) {
    should_process = 0;
}

/* Проверяем опцию -n (пропускать даты с переносом строки внутри) */
int has_newline = 0;
if (opts->single_line_only) {
    for (j = 0; j < 19; j++) {
        if (text[i + j] == '\n') {
            has_newline = 1;
            break;
        }
    }
}

if (should_process && !has_newline) {

    /* Выделяем найденную дату */
    if (opts->use_color) {
        /* Используем ANSI-цвет (синий) */
        fprintf(output, "%s", ANSI_BLUE);
        for (j = 0; j < 19; j++) {
            fputc(text[i + j], output);
        }
        fprintf(output, "%s", ANSI_RESET);
        i += 19;
    } else {
        /* Используем Markdown (курсив) */
        fputc('*', output);
        for (j = 0; j < 19; j++) {
            fputc(text[i + j], output);
        }
        fputc('*', output);
        i += 19;
    }
} else {
    /* Не выделяем, просто выводим символ */
    fputc(text[i], output);
    i++;
}
}

}

```

```
← → prg3aaaN3151.c inf2aaaN3151.S Makefile
prg3aaaN3151/prg3aaaN3151.c > int main( )
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  /* Структура для хранения параметров программы */
6  typedef struct {
7      int show_version; /* Флаг опции -v */
8      int use_color; /* Флаг опции -c */
9      int single_line_only; /* Флаг опции -n */
10     int begin_line; /* Начальная строка (-b), -1 если не задана */
11     int end_line; /* Конечная строка (-e), -1 если не задана */
12     char *input_file; /* Имя входного файла или NULL */
13     char *output_file; /* Имя выходного файла или NULL */
14 } Options;
15
16 /* Прототипы функций */
17 void print_version(void);
18 int parse_options(int argc, char *argv[], Options *opts);
19 int is_digit_char(char c);
20 int is_valid_date_time(const char *str, int pos);
21 void process_file(FILE *input, FILE *output, Options *opts);
22 void process_line(const char *line, FILE *output, Options *opts, int line_num);
23
24 /* Глобальные константы */
25 #define ANSI_BLUE "\033[34m"
26 #define ANSI_RESET "\033[0m"
27
28 int main(int argc, char *argv[]) {
29     Options opts = {0, 0, 0, -1, -1, NULL, NULL};
30     FILE *input = stdin;
31
32     FILE *output = stdout;
33     int exit_code = 0;
34
35     /* Парсим аргументы командной строки */
36     if (parse_options(argc, argv, &opts) != 0) {
37         return 1;
38     }
39
40     /* Если нужно показать версию - показываем и выходим */
41     if (opts.show_version) {
42         print_version();
43         return 0;
44     }
45
46     /* Открываем входной файл, если указан */
47     if (opts.input_file != NULL) {
48         input = fopen(opts.input_file, "r");
49         if (input == NULL) {
50             fprintf(stderr, "Ошибка: не удалось открыть файл %s\n", opts.input_file);
51             return 2;
52         }
53     }
54
55     /* Открываем выходной файл, если указан */
56     if (opts.output_file != NULL) {
57         output = fopen(opts.output_file, "w");
58         if (output == NULL) {
59             fprintf(stderr, "Ошибка: не удалось создать файл %s\n", opts.output_file);
60             if (input != stdin) fclose(input);
61             return 2;
62         }
63     }
64 }
```

```
← → prg3aaaN3151.c inf2aaaN3151.S Makefile
prg3aaaN3151/prg3aaaN3151.c
61     }
62 }
63
64 /* Обработываем файл */
65 process_file(input, output, &opts);
66
67 /* Закрываем файлы */
68 if (input != stdin) fclose(input);
69 if (output != stdout) fclose(output);
70
71 return exit_code;
72 }
73
74 /* Функция вывода информации о версии */
75 void print_version(void) {
76     printf("Студент: Агакарян Арам Арманович, гр. N3151\n");
77     printf("Вариант: 8-1-1-4\n");
78 }
79
80 /* Функция парсинга опций командной строки */
81 int parse_options(int argc, char *argv[], Options *opts) {
82     int i;
83     int file_arg_count = 0;
84
85     /* Проходим по всем аргументам */
86     for (i = 1; i < argc; i++) {
87         /* Если аргумент начинается с '-', это опция */
88         if (argv[i][0] == '-') {
89             if (strcmp(argv[i], "-v") == 0) {
90                 opts->show_version = 1;
91
92             } else if (strcmp(argv[i], "-c") == 0) {
93                 opts->use_color = 1;
94             } else if (strcmp(argv[i], "-n") == 0) {
95                 opts->single_line_only = 1;
96             } else if (strncmp(argv[i], "-b=", 3) == 0) {
97                 /* Парсим число после -b= */
98                 char *num_str = argv[i] + 3;
99                 char *endptr;
100                 long val = strtol(num_str, &endptr, 10);
101                 if (*endptr != '\0' || val < 1) {
102                     fprintf(stderr, "Ошибка: %s не является допустимым числом для -b\n", num_str);
103                     return 1;
104                 }
105                 opts->begin_line = (int)val;
106             } else if (strncmp(argv[i], "-e=", 3) == 0) {
107                 /* Парсим число после -e= */
108                 char *num_str = argv[i] + 3;
109                 char *endptr;
110                 long val = strtol(num_str, &endptr, 10);
111                 if (*endptr != '\0' || val < 1) {
112                     fprintf(stderr, "Ошибка: %s не является допустимым числом для -e\n", num_str);
113                     return 1;
114                 }
115                 opts->end_line = (int)val;
116             } else {
117                 fprintf(stderr, "Ошибка: неподдерживаемая опция %s\n", argv[i]);
118                 return 1;
119             }
120         } else {
121             /* Это не опция, значит имя файла */
122         }
123     }
124 }
```

```

← → prg3aaaN3151.c inf2aaaN3151.S Makefile
prg3aaaN3151/prg3aaaN3151.c
121     if (file_arg_count == 0) {
122         opts->input_file = argv[i];
123         file_arg_count++;
124     } else if (file_arg_count == 1) {
125         opts->output_file = argv[i];
126         file_arg_count++;
127     } else {
128         fprintf(stderr, "Ошибка: слишком много аргументов\n");
129         return 1;
130     }
131 }
132 }
133
134 /* Проверка корректности диапазона строк */
135 if (opts->begin_line != -1 && opts->end_line != -1) {
136     if (opts->begin_line > opts->end_line) {
137         fprintf(stderr, "Ошибка: начальная строка (%d) больше конечной (%d)\n",
138                 opts->begin_line, opts->end_line);
139         return 1;
140     }
141 }
142
143 return 0;
144 }
145
146 /* Проверка, является ли символ цифрой */
147 int is_digit_char(char c) {
148     return c ≥ '0' && c ≤ '9';
149 }
150

```

```

← → prg3aaaN3151.c inf2aaaN3151.S Makefile
prg3aaaN3151/prg3aaaN3151.c
181     if (!is_digit_char(str[i]) || !is_digit_char(str[i+1])) return 0;
182     int seconds = (str[i] - '0') * 10 + (str[i+1] - '0');
183     if (seconds > 59) return 0;
184     i += 2;
185
186     /* Пробел или перенос строки */
187     if (str[i] != ' ' && str[i] != '\n') return 0;
188     i++;
189
190     /* dd - день (01-31) */
191     if (!is_digit_char(str[i]) || !is_digit_char(str[i+1])) return 0;
192     int day = (str[i] - '0') * 10 + (str[i+1] - '0');
193     if (day < 1 || day > 31) return 0;
194     i += 2;
195
196     /* . */
197     if (str[i] != '.') return 0;
198     i++;
199
200     /* mm - месяц (01-12) */
201     if (!is_digit_char(str[i]) || !is_digit_char(str[i+1])) return 0;
202     int month = (str[i] - '0') * 10 + (str[i+1] - '0');
203     if (month < 1 || month > 12) return 0;
204     i += 2;
205
206     /* . */
207     if (str[i] != '.') return 0;
208     i++;
209
210     /* yyyy - год (4 цифры) */
211     if (!is_digit_char(str[i]) || !is_digit_char(str[i+1]) ||

```

```
← → prg3aaaN3151.c inf2aaaN3151.S Makefile
prg3aaaN3151/prg3aaaN3151.c
211     if (!is_digit_char(str[i]) || !is_digit_char(str[i+1]) ||
212         !is_digit_char(str[i+2]) || !is_digit_char(str[i+3])) return 0;
213
214     return 1; /* Формат корректен */
215 }
216
217 /* Обработка всего файла */
218 void process_file(FILE *input, FILE *output, Options *opts) {
219     char *buffer = NULL;
220     size_t total_read = 0;
221     char chunk[4096];
222     size_t bytes_read;
223
224     /* Читаем весь файл в буфер */
225     while ((bytes_read = fread(chunk, 1, sizeof(chunk), input)) > 0) {
226         char *new_buffer = realloc(buffer, total_read + bytes_read + 1);
227         if (new_buffer == NULL) {
228             free(buffer);
229             return;
230         }
231         buffer = new_buffer;
232         memcpy(buffer + total_read, chunk, bytes_read);
233         total_read += bytes_read;
234     }
235
236     if (buffer != NULL) {
237         buffer[total_read] = '\0';
238
239         /* Обработываем содержимое */
240         process_line(buffer, output, opts, 0);
241
242         free(buffer);
243     }
244 }
245
246 /* Обработка текста */
247 void process_line(const char *text, FILE *output, Options *opts, int unused) {
248     int i = 0;
249     int len = strlen(text);
250     int line_num = 1;
251
252     (void)unused; /* Параметр не используется */
253
254     while (i < len) {
255         /* Отслеживаем номер строки */
256         if (text[i] == '\n') {
257             line_num++;
258         }
259
260         /* Проверяем, начинается ли здесь дата-время */
261         /* Длина шаблона HH:MM:SS dd.mm.yyyy = 19 символов */
262         if (i + 19 ≤ len && is_valid_date_time(text, i)) {
263             /* Определяем, в каком диапазоне строк находится дата */
264             int date_line_start = line_num;
265             int j;
266
267             /* Проверяем опции -b и -e */
268             int should_process = 1;
269             if (opts->begin_line != -1 && date_line_start < opts->begin_line) {
270                 should_process = 0;
271             }
272         }
273     }
274 }
```

```
← → prg3aaaN3151.c inf2aaaN3151.S Makefile
prg3aaaN3151/prg3aaaN3151.c
271 }
272 if (opts->end_line != -1 && date_line_start > opts->end_line) {
273     should_process = 0;
274 }
275
276 /* Проверяем опцию -n (пропускать даты с переносом строки внутри) */
277 int has_newline = 0;
278 if (opts->single_line_only) {
279     for (j = 0; j < 19; j++) {
280         if (text[i + j] == '\n') {
281             has_newline = 1;
282             break;
283         }
284     }
285 }
286
287 if (should_process && !has_newline) {
288
289     /* Выделяем найденную дату */
290     if (opts->use_color) {
291         /* Используем ANSI-цвет (синий) */
292         fprintf(output, "%s", ANSI_BLUE);
293         for (j = 0; j < 19; j++) {
294             fputc(text[i + j], output);
295         }
296         fprintf(output, "%s", ANSI_RESET);
297         i += 19;
298     } else {
299         /* Используем Markdown (курсив) */
300         fputc('*', output);
301     }
302 }
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318 }

← → prg3aaaN3151.c inf2aaaN3151.S Makefile
prg3aaaN3151/prg3aaaN3151.c
292 fprintf(output, "%s", ANSI_BLUE);
293 for (j = 0; j < 19; j++) {
294     fputc(text[i + j], output);
295 }
296 fprintf(output, "%s", ANSI_RESET);
297 i += 19;
298 } else {
299     /* Используем Markdown (курсив) */
300     fputc('*', output);
301     for (j = 0; j < 19; j++) {
302         fputc(text[i + j], output);
303     }
304     fputc('*', output);
305     i += 19;
306 }
307 } else {
308     /* Не выделяем, просто выводим символ */
309     fputc(text[i], output);
310     i++;
311 }
312 } else {
313     /* Просто выводим символ */
314     fputc(text[i], output);
315     i++;
316 }
317 }
318 }
```


